

Validadores en JSF



Validadores en JSF

Los **validadores en JakartaServer Faces (JSF)** son una herramienta poderosa para garantizar que los datos ingresados por los usuarios cumplan con ciertos criterios antes de ser procesados. En esta lección, aprenderás sobre el concepto básico, su sintaxis y cómo implementarlos en un ejemplo práctico.

¿Qué son los Validadores en JSF?

Un validador en JSF verifica automáticamente que los datos ingresados en un formulario cumplan con reglas predefinidas antes de enviarlos al servidor. Esto se logra usando etiquetas específicas como `<f:validateLongRange>` o creando validadores personalizados.

Características Clave de los Validadores

1. **Validación Automática:** Se realiza antes de que los datos sean procesados por el backend.
2. **Mensajes de Error Personalizados:** JSF permite mostrar mensajes específicos para cada validación fallida.
3. **Flexibilidad:** Puedes usar validadores predefinidos o crear los tuyos.

Sintaxis Básica de un Validador

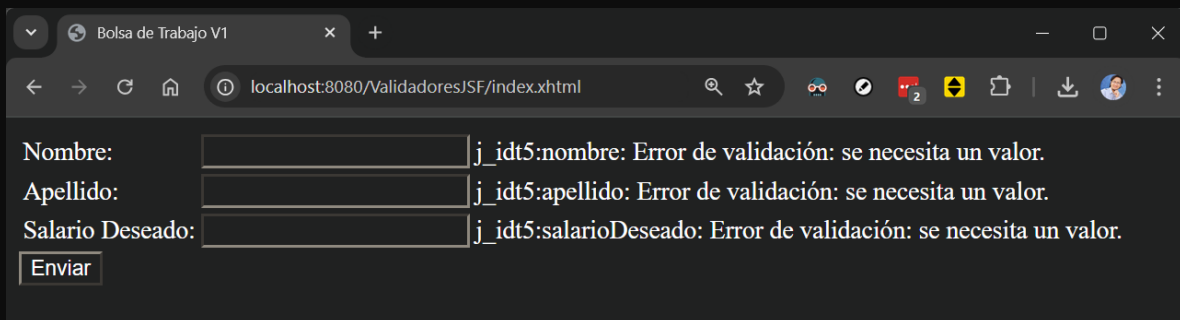
La validación en JSF se define dentro de los componentes de entrada (<h:inputText>, <h:inputTextarea>, etc.) utilizando etiquetas como:

```
<f:validateLongRange minimum="500" maximum="5000"/>
```

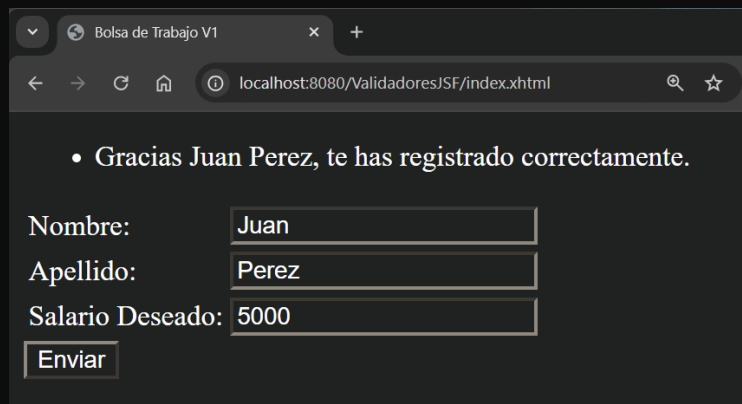
Este ejemplo asegura que el valor ingresado esté entre 500 y 5000.

Ejemplo Práctico

Resultado Esperado:



A screenshot of a web browser window titled "Bolsa de Trabajo V1" showing a registration form at the URL "localhost:8080/ValidadoresJSF/index.xhtml". The form has three input fields: "Nombre:", "Apellido:", and "Salario Deseado:". Each field has a corresponding error message to its right: "j_idt5:nombre: Error de validación: se necesita un valor.", "j_idt5:apellido: Error de validación: se necesita un valor.", and "j_idt5:salarioDeseado: Error de validación: se necesita un valor." respectively. There is an "Enviar" button at the bottom left of the form.



A screenshot of a web browser window titled "Bolsa de Trabajo V1" showing a successful registration message at the URL "localhost:8080/ValidadoresJSF/index.xhtml". The message is "• Gracias Juan Perez, te has registrado correctamente." Below the message, the registration form is shown with the following values: "Nombre:" Juan, "Apellido:" Perez, and "Salario Deseado:" 5000. There is an "Enviar" button at the bottom left of the form.

Archivo index.xhtml

En este archivo, creamos un formulario que valida el nombre, apellido y salario deseado de un candidato:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
<h:head>
  <title>Bolsa de Trabajo V1</title>
</h:head>
<h:body>
```

```

<h:form>
  <h:messages globalOnly="true"></h:messages>
  <table>
    <tr>
      <td><h:outputLabel for="nombre" value="Nombre:"></h:outputLabel></td>
      <td><h:inputText id="nombre" required="true"
        value="#{candidato.nombre}"></h:inputText></td>
      <td><h:message for="nombre"></h:message></td>
    </tr>
    <tr>
      <td><h:outputLabel for="apellido" value="Apellido:"></h:outputLabel></td>
      <td><h:inputText id="apellido" required="true"
        value="#{candidato.apellido}"></h:inputText></td>
      <td><h:message for="apellido"></h:message></td>
    </tr>
    <tr>
      <td><h:outputLabel for="salarioDeseado"
        value="Salario Deseado:"></h:outputLabel></td>
      <td>
        <h:inputText id="salarioDeseado" required="true"
          value="#{candidato.salarioDeseado}"
          <f:validateLongRange minimum="500" maximum="5000"/>
        </h:inputText>
      </td>
      <td><h:message for="salarioDeseado"></h:message></td>
    </tr>
  </table>
  <h:commandButton action="#{vacanteForm.enviar}" value="Enviar" />
</h:form>
</h:body>
</html>

```

Explicación del Código

1. Etiqueta <h:inputText> con Atributo required:

Asegura que el campo no se envíe vacío.

```

<h:inputText id="nombre" required="true"
value="#{candidato.nombre}"></h:inputText>

```

2. Etiqueta <f:validateLongRange>:

Valida que el salario deseado esté en el rango de 500 a 5000.

```

<f:validateLongRange minimum="500" maximum="5000"/>

```

3. Mensajes de Error Personalizados con `<h:message>`:

Muestra errores específicos junto a cada campo cuando no se cumplen las validaciones.

```
<h:message for="nombre"></h:message>
```

4. Mensajes Globales con `<h:messages>`:

Muestra mensajes generales que no están asociados a un campo específico.

```
<h:messages globalOnly="true"></h:messages>
```

Código del Modelo `Candidato.java`

El modelo almacena los datos del formulario y registra los cambios con un `Logger`:

```
package beans.model;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("candidato")
@SessionScoped
public class Candidato implements Serializable {

    private final Logger log = LoggerFactory.getLogger(Candidato.class);
    private String nombre;
    private String apellido;
    private String salarioDeseado;

    public Candidato() {
        log.info("Creando el objeto Candidato");
        this.nombre = "Introduce tu nombre";
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
        log.info("Nombre modificado: " + nombre);
    }
}
```

```
public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
    log.info("Apellido modificado: " + apellido);
}

public String getSalarioDeseado() {
    return salarioDeseado;
}

public void setSalarioDeseado(String salarioDeseado) {
    this.salarioDeseado = salarioDeseado;
    log.info("Salario deseado modificado: " + salarioDeseado);
}
}
```

Código del Controlador `vacanteForm.java`

Este controlador procesa el formulario y verifica si los datos ingresados son válidos:

```
package beans.backing;

import beans.model.Candidato;
import jakarta.enterprise.context.RequestScoped;
import jakarta.faces.application.FacesMessage;
import jakarta.faces.context.FacesContext;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

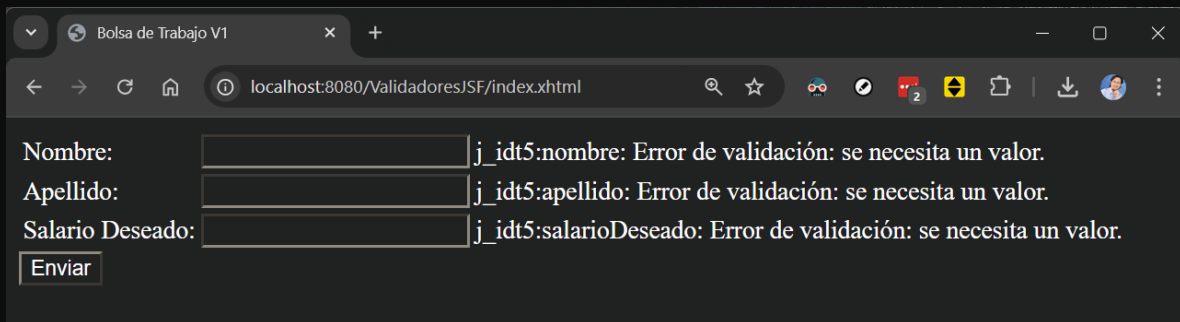
@Named("vacanteForm")
@RequestScoped
public class VacanteForm {
    private final Logger log = LoggerFactory.getLogger(VacanteForm.class);

    @Inject
    private Candidato candidato;

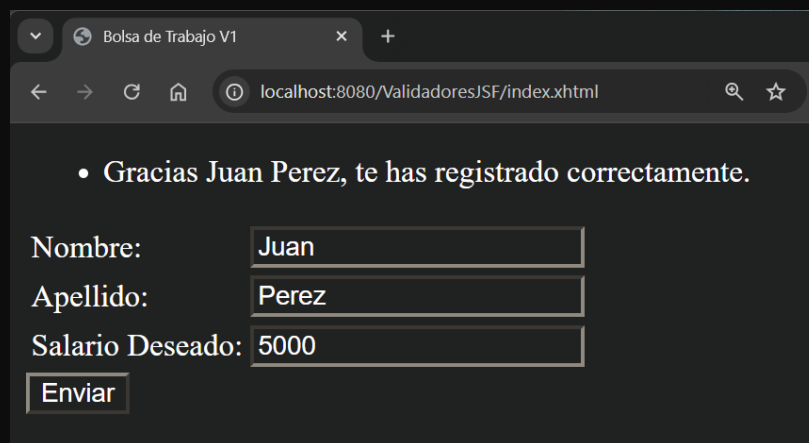
    public String enviar() {
```

```
        if ("Juan".equals(candidato.getNombre()) &&
            "Perez".equals(candidato.getApellido())) {
            String msg = "Gracias Juan Perez, te has registrado correctamente.";
            FacesMessage facesMessage = new FacesMessage(FacesMessage.SEVERITY_ERROR,
msg, null);
            FacesContext.getCurrentInstance().addMessage(null, facesMessage);
            log.info("Usuario registrado correctamente");
            return "index"; // Redirige a la misma página
        }
        log.info("Datos de usuario inválidos");
        return "fallo"; // Redirige a una página de éxito
    }
}
```

Resultado:



A screenshot of a web browser window titled "Bolsa de Trabajo V1". The address bar shows "localhost:8080/ValidadoresJSF/index.xhtml". The form contains three input fields: "Nombre:", "Apellido:", and "Salario Deseado:". Each field has a corresponding error message to its right: "j_idt5:nombre: Error de validación: se necesita un valor.", "j_idt5:apellido: Error de validación: se necesita un valor.", and "j_idt5:salarioDeseado: Error de validación: se necesita un valor." respectively. Below the fields is an "Enviar" button.



A screenshot of a web browser window titled "Bolsa de Trabajo V1". The address bar shows "localhost:8080/ValidadoresJSF/index.xhtml". At the top, there is a success message: "• Gracias Juan Perez, te has registrado correctamente." Below this, the form fields are filled: "Nombre:" with "Juan", "Apellido:" with "Perez", and "Salario Deseado:" with "5000". The "Enviar" button is still present at the bottom.

Conclusión

Este ejemplo muestra cómo usar validadores en JSF para garantizar la integridad de los datos antes de procesarlos. Además, los mensajes de error proporcionan una experiencia de usuario clara y amigable.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)